RADAR

Plataforma de Scoring de Riesgo de Cuentas y Apoyo a Decisiones de Crédito

Documentación Técnica · v1.0

DynamicAI · 2026

Radar — Documentación Técnica de la A a la Z

🎯 Personalizable para su Empresa

Radar es una plataforma de código abierto (AGPL-3.0) y es **personalizable según las reglas de negocio, el perfil sectorial y el apetito de riesgo de cada empresa**. Las fórmulas de scoring, ponderaciones, factores de riesgo sectorial, supuestos macro (interés, inflación), tramos de plazo y lógica de decisión son totalmente configurables.

Contacte con DynamicAI para **integración empresarial, desarrollo de módulos a medida, despliegue on-premise y formación**: *osmanedembele6361@gmail.com*

> Plataforma de scoring de riesgo de cuentas y apoyo a decisiones de crédito. Última actualización: 2026-05-03

---

1. Resumen del Proyecto

Radar es una aplicación de **scoring de crédito / evaluación de riesgo de cuentas por cobrar** dirigida a pymes y empresas corporativas. Se posiciona como la contraparte interna (privada) de un servicio de calificación tipo Findeks.

Flujo principal: 1. El usuario crea un cliente (cuenta) 2. Introduce datos de antigüedad (aging) y solicitud de crédito, o los importa por lotes desde Excel 3. El sistema produce una **calificación crediticia** basada en disciplina de pago, ratios financieros (Z-Score, F-Score, DSCR, ICR), factores de riesgo sectorial y condiciones macro 4. Simulación Monte Carlo + pruebas de estrés con nombre calculan la distribución de escenarios 5. Se presenta un informe PDF/HTML detallado y multilingüe (TR/EN/ES/DE)

---

2. Stack Tecnológico

Capa	Tecnología
Backend	Python 3.11, **Flask 3.0** (Blueprints)
ORM	**SQLAlchemy 2.0** (scoped_session)
BD	PostgreSQL (prod) / SQLite (dev) — doble destino
Auth	flask-login + **Authlib** (Google OAuth) + itsdangerous (verificación por mail)
Mail	flask-mail (SMTP)
i18n	**flask-babel 4.x** (patrón locale_selector)
Templating	Jinja2 + Bootstrap 5.3 + Bootstrap Icons
API Docs	**Flasgger** (OpenAPI/Swagger)
Async	Celery + Redis (planificado, aún no activo)
Container	Docker + docker-compose, `network_mode: host`
WSGI	gunicorn 4 workers
Reverse Proxy	nginx — `technodai.com/radar/` → `127.0.0.1:5001`
Excel	pandas + openpyxl

3. Arquitectura de Alto Nivel

```
┌──────────────────────────────────────────────────────┐
│  Browser (TR/EN/ES/DE)                               │
└──────────────────────────────────────────────────────┘
                        │
                        │ https://technodai.com/radar
                        ▼
                  ┌──────────┐
                  │  nginx   │  TLS, prefix routing
                  └──────────┘
                       │
                       │ proxy_pass :5001
                       ▼
              ┌──────────────────┐
              │  gunicorn × 4    │  Docker (host net)
              │   ↓              │
              │  Flask app       │
              │   ├ blueprints/  │  auth, main, customer,
              │   │              │  scoring, admin, api
              │   ├ enterprise/  │  i18n, RBAC, audit, webhooks
              │   └ analytics/   │
              └──────────────────┘
                        │
                        │
            ┌───────────┴───────────┐
            ▼          ▼          ▼
       PostgreSQL   SMTP      Google OAuth
       (advisory   (verify     (login)
        locks)     mails)
```

---

4. Mapa de Módulos

4.1 Núcleo

Archivo	Responsabilidad
`app.py`	Flask factory (`create_app`), registro de blueprints, `init_db()` , init de funciones empresariales
`database.py`	Modelos SQLAlchemy, motor dual-DB, migración ALTER ligera, **Postgres advisory lock** (seguridad ante concurrencia)
`extensions.py`	`login_manager` , `mail` , `ts` (URLSafeTimedSerializer) — singletons compartidos a nivel de módulo
`credit_scoring.py`	Clase `CreditScorer` — cálculo de score, notas de evaluación experta, Monte Carlo + pruebas de estrés con nombre
`aging_analyzer.py`	Construye métricas de retraso a partir de registros de aging
`excel_import.py`	Importación masiva de clientes/registros (pandas)
`currency.py`	Formato TRY, ayudantes de conversión FX
`logger.py`	Logging estructurado
`translations.py`	Diccionario en 4 idiomas (TR/EN/ES/DE) — dict plano, ~70 KB

4.2 Rutas (Blueprints)

Blueprint	Ruta	Responsabilidad
`auth`	`/login` , `/register` , `/logout` , `/google-login` , `/verify/<token>`	Autenticación de usuarios
`main`	`/` , `/nedir` , `/guvenlik` , `/set_language/<lang>`	Inicio, contenido estático, cambio de idioma
`customer`	`/musteri/...`	CRUD + detalle
`scoring`	`/skor/...` , `/rapor/<id>`	Cálculo de score, render de informe
`admin`	`/admin/...`	Configuración, gestión de usuarios, audit log
`api`	`/api/v1/...`	Endpoints REST (documentados con Flasgger)

4.3 Capa Enterprise (Fase 8-11)

Archivo	Responsabilidad
`enterprise.py`	`init_enterprise_features(app)` — RBAC, audit, init multi-tenant
`i18n_utils.py`	Selector de locale flask-babel 4.x
`analytics.py`	Agregación de métricas a nivel de tenant
`api_docs.py`	Configuración de Swagger UI con Flasgger
`webhooks.py`	Envío de webhooks salientes (Slack, Discord, custom)

---

5. Modelo de Datos

```
Tenant 1──* User 1──* Customer 1──* AgingRecord
                                  │
                                  ├──* CreditRequest
                                  └──* CreditScore (history)

User *──* Role  (UserRole pivot)
Tenant 1──* AuditLog
```

Importante: - `tenant_id` es `nullable=True default=1` en cada tabla — los registros antiguos caen al tenant por defecto - Los clientes con `is_sample=True` son visibles para todos los usuarios (datos demo) - `CreditScore` está versionado — cada cálculo crea una nueva fila, el historial nunca se borra

---

6. Arquitectura de Despliegue

Producción (technodai.com)

```
~/apps/radar/                # git clone
├── docker-compose.yml       # binding de puerto local-only (5001)
├── .env                     # SECRET_KEY, DB_URL, OAuth, SMTP
└── data/                    # SQLite fallback + uploads (volumen)
```

- Container `radar_v1_app`, `restart: always`, host network
- BD `radar` en Postgres, `pg_advisory_lock(7382001)` evita la condición de carrera de init de los 4 workers
- vhost de nginx: prefijo `/radar/` con `proxy_pass http://127.0.0.1:5001/`

Local (dev)

- Mismo compose file (symlink en Desktop), puerto `8005`
- Cae a SQLite, funciona sin BD aparte

Flujo de Despliegue

```
edición local
   ↓ git commit + push
   ↓ ssh prod
cd ~/apps/radar && git pull
sudo docker compose up -d --build
   ↓ smoke test
curl -I https://technodai.com/radar/
```

---

7. Workflow de Desarrollo

1. **Branch**: commits pequeños sobre `main`. Topic branches para features grandes.
2. **Migración**: `database.py:_run_lightweight_migrations()` usa el patrón `ALTER TABLE IF NOT EXISTS` — Alembic deliberadamente evitado para el esquema pequeño.
3. **Seed**: `seed_sample_data.py` — set de clientes demo para nuevas instalaciones.
4. **Tests**: `tests/security_smoke.py` corre en CI; `integrity_check.py` es manual.
5. **i18n**: Las nuevas cadenas deben añadirse en los 4 bloques de `translations.py`. CI fuerza la paridad de claves.
6. **Mensaje de commit**: `<type>(<scope>): <resumen>` — `feat`, `fix`, `chore`, `refactor`.

---

8. Flujo del Usuario Final

Paso	Página	Acción
1	`/register`	Registro por email/contraseña o Google → verificación por mail
2	`/`	Lista de clientes — añadir nuevo cliente
3	`/musteri/yeni`	Información de la cuenta (sector, patrimonio, ingresos, etc.)
4	`/musteri/<id>`	Añadir fila de aging + solicitud de crédito
5	`/musteri/<id>/import`	Importación masiva por Excel
6	`/skor/<id>`	Calcular score — se aplican los ajustes del admin (interés, inflación)
7	`/rapor/<id>`	Informe detallado — evaluación ejecutiva, escenarios, pruebas de estrés con nombre
8	`/admin`	Configuración, gestión de usuarios, audit log

---

9. Roadmap — Próximos Pasos

9.1 Corto Plazo (1-2 semanas)

- [ ] **Expansión del módulo de plazos**: hoy plazos escalonados estáticos (`vade_15..90`); cálculo dinámico por banda de disciplina de pago (DSO + mediana sectorial)
- [ ] **Exportación PDF**: integración WeasyPrint para `?format=pdf` en el informe
- [ ] **Esqueleto de tests unitarios**: `pytest` + `pytest-flask` , ≥70% coverage en `credit_scoring`
- [ ] **UI del audit log**: tabla filtrable dentro del panel admin

9.2 Medio Plazo (1-2 meses)

- [ ] **Activación de Celery**: import Excel + score por lotes fuera del hilo de la request (los archivos grandes hacen timeout hoy)
- [ ] **Expansión de eventos webhook**: `score.calculated` , `customer.high_risk` , `request.spike`
- [ ] **Rate limiting (API por tenant)**: cuota Redis activa para login/register; cuota por tenant en `/api/v1/*` queda abierta
- [ ] **Feed de benchmark sectorial**: benchmark público de ratios sectoriales (KGK / TÜİK / Findeks) — hoy solo mediana de pares
- [ ] **Canales de notificación**: email + Slack + Discord — alertas de alto riesgo
- [ ] **Multi-divisa**: hoy todo es TRY — tabla snapshot FX para cuentas USD/EUR

9.3 Largo Plazo (3+ meses)

- [ ] **Capa ML**: score basado en reglas + overlay opcional de regresión logística / gradient boosting (con suficientes registros)
- [ ] **Open Banking**: pull de movimientos de cuenta → aging automático
- [ ] **App móvil**: PWA o React Native — orientada a lectura (vista de score, alertas)
- [ ] **Onboarding self-serve de tenant**: SaaS marketplace (hoy la creación de tenant es manual)
- [ ] **Panel GDPR/KVKK**: exportación de datos, solicitud de borrado, política de retención
- [ ] **2FA**: TOTP (pyotp) — especialmente para roles admin
- [ ] **Exportación de audit log**: stream JSON para SIEM (Splunk, ELK)

---

10. Mejora Continua — Refuerzos Aplicados

Tema	Mejora	Estado
Campo `version` obsoleto en docker-compose	Campo eliminado	✅
Sin reintento de webhook	`_post_with_retry` — 3 intentos + backoff exponencial (1s/2s/4s); reintento en 5xx y errores de red, 4xx un solo intento	✅
Sin ID de correlación en logs	Middleware `request_id` — UUID16 por request, header `X-Request-ID` + binding `g.request_id`	✅
Borrado físico del audit log	Columna `AuditLog.deleted_at` — patrón soft delete (retención de cumplimiento)	✅
Sin automatización de backup BD	`scripts/backup.sh` — autodetecta Postgres/SQLite, gzip, rotación de retención; **cron prod 03:00**	✅
Flask 3.0.0 CVE-2026-27205	Pin `flask>=3.1.3` + upgrade de pip/wheel/setuptools en CI	✅
Rate limiter `memory://` se dividía entre workers	`LIMITER_STORAGE_URI=redis://localhost:6379/1` — contador compartido, verificado en prod (la 11ª contraseña incorrecta devolvió 429)	✅
Evaluación Ejecutiva congelada en TR al reabrir el informe en otro idioma	`assessment_i18n` / `decision_summary_i18n` — almacenado como dict JSON de 4 idiomas; la vista cae al idioma activo; filas legacy compatibles	✅
Import Excel no asignaba `user_id` a nuevas filas `Customer`	Asignación de `current_user.id` — resuelta la pérdida de propietario	✅
`Carga de Excel` aparecía en el menú a usuarios no-admin (al hacer click redirigía)	Flag `is_admin` en `context_processor` + gating `{% if is_admin %}` ; enlace `Panel` redundante eliminado	✅
Sin alerta en 5xx	`errorhandler(500)` → mail a `ADMIN_EMAIL` con request_id + path + user + resumen de la excepción	✅

Abierto (en el roadmap)

Tema	Impacto	Plan
Sin Alembic	El ALTER ligero alcanza; revisar si crece el esquema	Cuando el esquema cruce las 30+ tablas
Cobertura de tests baja	Riesgo en refactor	Esqueleto pytest listo, objetivo 50% en paths críticos
`translations.py` un solo archivo de 70 KB	Imán de merge conflicts	Un archivo por idioma + lazy load
Authlib deprecation (`authlib.jose`)	Romperá en 2.0	Migrar a `joserfc`
docker-compose override en el servidor	`git stash` cada pull	Separar en `docker-compose.override.yml`

---

11. Notas de Seguridad — Controles Activos

Identidad y Sesión

- ✅ Contraseñas cifradas con werkzeug PBKDF2-SHA256 (con sal)
- ✅ Verificación por mail obligatoria (URLSafeTimedSerializer, con caducidad)
- ✅ Google OAuth — validación de state Authlib (protección CSRF)
- ✅ Flags de cookie: `HttpOnly` , `SameSite=Lax` , `Secure` en prod

Capa Web (flask-wtf, flask-limiter, flask-talisman)

- ✅ **Protección CSRF**: `csrf_token` en cada formulario POST; blueprint API exento (basado en token)
- ✅ **Rate limiting**: `/login` 10/min + 50/h, `/register` 5/min + 20/h; default 100/min + 1000/h. **Backend prod Redis** (`LIMITER_STORAGE_URI=redis://localhost:6379/1`) — contador compartido entre workers.
- ✅ **Cabeceras de seguridad** (Talisman): HSTS (1 año), `X-Frame-Options: SAMEORIGIN` , `X-Content-Type-Options: nosniff` , `Referrer-Policy: strict-origin-when-cross-origin`
- ✅ **Request ID**: cada request lleva un header `X-Request-ID` → correlación de logs
- ✅ **Gating admin de menú**: el flag `is_admin` oculta de usuarios no-admin los links que requieren `@admin_required`

Acceso a Datos

- ✅ Aislamiento multi-tenant — toda query filtra por `tenant_id`

- ✅ ORM SQLAlchemy — queries parametrizadas (protección SQL injection)
- ✅ El flag `is_sample` no salta el aislamiento de tenant
- ✅ Postgres advisory lock — protección contra race de init concurrente
- ✅ Soft-delete en audit log — sin borrado físico, retención de cumplimiento

Operacional

- ✅ Secretos en `.env`, nunca en repo (`.gitignore` + 8 tests de seguridad)
- ✅ Script de backup BD + rotación de retención (`scripts/backup.sh`) — **cron prod cada noche 03:00**
- ✅ Dependencias pinneadas, `pip-audit --strict` en CI (incluye bump del CVE de Flask)
- ✅ Tope de upload 10 MB (reducción de superficie DoS)
- ✅ **Alerta admin 5xx**: `errorhandler(500)` envía request_id + excepción a `ADMIN_EMAIL`

Roadmap (Abierto)

- ⚠️ 2FA (TOTP) — especialmente para roles admin
- ⚠️ Firma HMAC saliente de webhook obligatoria por defecto
- ⚠️ CSP (Content Security Policy) — hoy hay style/script inline, requiere refactor
- ⚠️ Migración Authlib `joserfc` (deprecation 2.0)

---

12. Comandos Operacionales

```
# Dev local
python app.py                                    # SQLite

# Docker (puerto local 8005)
docker compose up -d --build
docker compose logs -f radar-app

# Despliegue prod (desde máquina local)
sshpass -p "$SERVER_PASS" ssh "$SERVER_USER@$SERVER_IP" \
  "cd ~/apps/radar && git pull && \
   echo \$PASS | sudo -S docker compose up -d --build"

# Smoke test
curl -I https://technodai.com/radar/

# Backup BD (en prod)
sudo docker exec radar_v1_app pg_dump -U radar radar > backup_$(date +%F).sql

# Migración (ligera, corre automáticamente en init_db)
python -c "from database import init_db; init_db()"

# Seed
python seed_sample_data.py

# Paridad de claves de traducción (manual)
python -c "from translations import translations; \
  ks = {l: set(translations[l]['expert_assessments'].keys()) for l in translations}
  print('diff:', ks['tr'] ^ ks['en'])"
```

13. Registros de Decisión (mini-ADR)

1. **ALTER ligero en lugar de Alembic**: esquema pequeño, downtime aceptable. Revisar cuando crezca la complejidad.
2. `network_mode: host` : mismo host que nginx, sin complejidad de port mapping. Cambiará para multi-host.
3. **flask-babel 4.x**: Flask 3.0 incompatible con 3.x (`locked_cached_property` eliminado).
4. **Tenant único por defecto ID=1**: filas legacy retropobladas; transición multi-tenant sin downtime.
5. **Clave de advisory lock Postgres 7382001**: número mágico fijo — todos los workers piden el mismo lock, el primero crea el esquema, el resto espera.
6. **Pill de sponsor en el footer**: el navbar no tenía espacio, se mantuvo la visibilidad llevándolo al footer.

14. Licencia y Open Source

- **Licencia del proyecto:** AGPL-3.0 — quien lo aloje como servicio debe compartir el código fuente
- **Dependencias:** todas permisivas (MIT/BSD/Apache); sin GPL/AGPL → compatibles con AGPL
- **Dependencias destacables:**
- `psycopg2-binary` (LGPL) — linking dinámico, sin problema
- `certifi` (MPL-2.0) — copyleft a nivel de archivo, OK mientras no lo modifiquemos
- **Auditoría de licencias:** hecha con `pip-licenses`, salida limpia

15. Test y Calidad

Tests existentes

- `tests/security_smoke.py` — 8 chequeos de seguridad:
- `.env` y secretos no están en git
- Sin archivos de claves privadas
- Sin SECRET/PASSWORD/API_KEY hardcodeados
- Paquetes críticos pinneados (flask, sqlalchemy, authlib, itsdangerous, flasgger)
- Todas las rutas protegidas envueltas con `@login_required`
- SECRET_KEY definido y largo
- Archivo LICENSE es AGPL-3.0
- `tests/integrity_check.py` — validación manual de score (requiere BD)

CI (GitHub Actions)

- `.github/workflows/ci.yml` — en cada push/PR:
- suite de seguridad pytest
- escaneo de vulnerabilidades de dependencias `pip-audit --strict`
- paridad de claves de traducción (TR/EN/ES/DE)

Ejecución

```
pytest tests/ -v                    # todos los tests
pytest -m security                  # solo smoke de seguridad
pip-audit                           # escaneo CVE
```

16. Referencias

- Repo: `git@github.com:Dai-Solutions/Radar.git`
- Prod: https://technodai.com/radar/
- Sponsor: OpenCollective + Patreon (`https://www.patreon.com/c/DynamicAI`)
- Findeks (competidor): https://www.findeks.com/